

MODULE 1 ASSIGNMENT

"How the Internet Delivers a Web Page"

Student Name:	Amaljith M	Course:	PCCST501 – Computer Networks
Roll No:	10	Module:	1
Class:	CSA	Website Chosen:	YouTube (www.youtube.com)

When a user types **www.youtube.com** into a web browser's address bar and presses **Enter**, a highly orchestrated sequence of networking operations unfolds across all layers of the Internet protocol suite—from resolving the domain name to establishing secure transport channels and loading multimedia web resources. This report explains the end-to-end process in detail.

1. The TCP/IP Protocol Stack and the Role of Each Layer

Communication over the Internet is organized using the **TCP/IP protocol stack**, which divides the process of moving network data into four distinct layers. Each layer performs a specific task and passes data to the adjacent layer via encapsulation and decapsulation.

Application Layer

Protocols: HTTP/HTTPS, DNS, QUIC, TLS | Formulates requests (HTTP GET) and parses response payloads (HTML/CSS/JS).

Transport Layer

Protocols: TCP, UDP | Provides process-to-process communication, segmentation, sequence numbering, and reliable delivery.

Internet Layer

Protocols: IPv4, IPv6, ICMP | Handles logical IP addressing and packet routing across intermediate networks and routers.

Network Access Layer

Protocols: Ethernet (802.3), Wi-Fi (802.11), ARP | Converts IP packets into physical frames transmitted over physical media.

Figure 1: The four layers of the TCP/IP protocol stack

- **Application Layer:** Where the browser (e.g., Chrome) and YouTube web application operate. It generates the HTTP/HTTPS requests to fetch web assets (HTML, CSS, JS, video thumbnails) and interprets the server responses. DNS resolution also functions at this layer.
- **Transport Layer:** Manages end-to-end communications between the user application and YouTube's servers using port numbers (e.g., Port 443 for HTTPS). TCP guarantees ordered, error-checked delivery of HTTP segments via sequence numbers and acknowledgments.
- **Internet Layer:** Encapsulates transport segments into IP packets, attaching source and destination IP addresses. Routers use IP headers to forward packets across the Internet until they reach Google/YouTube edge network nodes.
- **Network Access Layer:** Converts IP packets into physical data frames (Ethernet/Wi-Fi) appropriate for the local physical medium (fiber, copper cable, or radio signals) to transport data across local network links.

2. Role of DNS in Domain Resolution

Humans identify web services using human-friendly domain names like `www.youtube.com`, whereas routers and servers require IP addresses (e.g., `142.250.190.46`). The **Domain Name System (DNS)** translates domain names into numerical IP addresses.

Step-by-step resolution process for YouTube:

1. **Browser & OS Cache Lookup:** The browser first checks its internal DNS cache. If not found, it queries the OS DNS cache and local hosts file.
2. **Recursive Resolver:** If missing locally, the request is forwarded to the ISP's DNS resolver or a public resolver (e.g., Google DNS `8.8.8.8`).
3. **Root Name Server Query:** The resolver contacts a Root DNS Server, which redirects it to the Top-Level Domain (TLD) server for `.com`.
4. **TLD Name Server Query:** The `.com` TLD server directs the resolver to the Authoritative Name Server responsible for `youtube.com`.
5. **Authoritative Server Response:** The Authoritative Server returns the corresponding **A Record** (IPv4) or **AAAA Record** (IPv6) containing YouTube's edge IP address.
6. **Caching:** The IP address is saved in local caches according to its TTL (Time To Live) so subsequent requests bypass DNS lookup overhead.

3. How HTTP/HTTPS Requests and Receives the Webpage

Once the browser resolves YouTube's IP address, it uses **HTTPS** (HTTP over TLS/SSL) to communicate securely with YouTube servers.

- **TLS Handshake:** Before sending HTTP requests, a TLS handshake is performed over TCP Port 443 to authenticate YouTube's digital certificate and negotiate symmetric encryption keys.
- **Client Request:** The browser issues an encrypted **HTTP GET** request:

```
GET / HTTP/1.1
Host: www.youtube.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: text/html,application/xhtml+xml
Accept-Language: en-US,en;q=0.9
```

- **Server Response:** YouTube's frontend servers process the request and respond with an **HTTP 200 OK** status along with the HTML document payload:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Set-Cookie: VISITOR_INFO1_LIVE=...; Secure; HttpOnly

<!DOCTYPE html><html>...</html>
```

- **DOM Rendering & Auxiliary Requests:** As the browser parses the received HTML DOM tree, it encounters linked resources (CSS stylesheets, JavaScript files, images, video stream manifests) and initiates secondary parallel HTTPS requests to fetch them.

4. How TCP Provides Reliable Communication for HTTP

HTTP depends on **TCP** at the transport layer to ensure that all transmitted web elements arrive without loss or corruption.

- **Three-Way Handshake:** TCP establishes a reliable connection before data transfer:
 1. **SYN:** Browser sends a SYN segment with an initial sequence number ($\text{\$Seq} = X$).
 2. **SYN-ACK:** YouTube server responds with SYN-ACK ($\text{\$Seq} = Y$, $\text{\$Ack} = X + 1$).
 3. **ACK:** Browser completes the setup with ACK ($\text{\$Ack} = Y + 1$).

- **Flow & Congestion Control:** TCP regulates transmission speeds using sliding window mechanisms and congestion control algorithms (e.g., CUBIC or BBR) to avoid saturating network links or overflowing receiver buffers.
- **Error Detection & Retransmission:** Each segment includes a checksum. If a segment is dropped or corrupted, the receiving side fails to send an ACK, prompting the sender to trigger a retransmission via Automatic Repeat reQuest (ARQ).

5. Application Architecture: Client-Server vs. Peer-to-Peer

YouTube operates on a **Client-Server Architecture**.

Justification:

- **Centralized Infrastructure:** YouTube hosts its databases, user profiles, video processing backend, and Content Delivery Networks (CDNs) on Google's centralized cloud infrastructure.
- **Asymmetric Roles:** The web browser acts purely as a *Client* requesting page layouts and video streams, while Google's infrastructure acts as the *Server* processing queries and serving media.
- **No Direct Peer Sharing:** Users do not download or upload video stream segments directly to/from other active viewers (unlike P2P networks like BitTorrent); all content is fetched from Google edge servers.

6. Comparison Between HTTP and FTP

Feature / Characteristic	HTTP (Hypertext Transfer Protocol)	FTP (File Transfer Protocol)
Primary Purpose	Designed for transferring hypertext documents, web pages, APIs, and web media.	Designed specifically for uploading and downloading bulk files between machines.
Connection Model	Stateless protocol (each request/response pair is independent without session cookies).	Stateful protocol (maintains an active control connection throughout the session).
Port Numbers	Uses Port 80 (HTTP) or Port 443 (HTTPS).	Uses Port 21 for Control and Port 20 for Data transfer.
Data Channel Design	Transmits both control commands and payload data over a single TCP connection.	Uses two separate connections: Control Channel (Port 21) and Data Channel (Port 20).

7. How Cookies Improve User Experience

A **Cookie** is a small piece of data sent by a web server and stored locally on the user's browser. Because HTTP is inherently stateless, cookies enable websites to track session state across multiple page requests.

Real-World YouTube Example: When you log into YouTube, the server authenticates your credentials and issues a cookie containing a unique session token (e.g., LOGIN_INFO). As you navigate YouTube or open new tabs, your browser attaches this cookie to every request. This keeps you logged in, retains your customized subscriptions, preserves Dark Mode settings, and remembers video playback progress and volume preferences without requiring re-authentication on every page reload.

8. End-to-End Communication Flow Diagram



9. Conclusion

Application-layer protocols play a fundamental role in modern Internet usage by translating user actions into structured network communications. When loading a platform like YouTube, protocols such as DNS translate readable web addresses into IP endpoints, while HTTP/HTTPS ensures secure transmission of web pages and multimedia content. Combined with session cookies that bypass HTTP's stateless nature to maintain user login states and preferences, and underlying transport protocols like TCP that guarantee packet delivery, application-layer protocols create a smooth, reliable, and secure web browsing experience for millions of users daily.